

Exercise 6 - Cross Validation of Models

Florescu Andrei - 12242987

2024

Task 1

```
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.4.2

library(knitr)

## Warning: package 'knitr' was built under R version 4.4.2

data(Auto)
set.seed(12242987)

kable(head(Auto), caption = "Auto dataset (head)")
```

Table 1: Auto dataset (head)

mpg	cylinders	displacement	horsepower	weight	acceleration	year	origin	name
18	8	307	130	3504	12.0	70	1	chevrolet chevelle malibu
15	8	350	165	3693	11.5	70	1	buick skylark 320
18	8	318	150	3436	11.0	70	1	plymouth satellite
16	8	304	150	3433	12.0	70	1	amc rebel sst
17	8	302	140	3449	10.5	70	1	ford torino
15	8	429	198	4341	10.0	70	1	ford galaxie 500

1. Fit multiple models

```
# fit models
model1 <- lm(mpg ~ horsepower, data = Auto)
model2 <- lm(mpg ~ poly(horsepower, 2), data = Auto)
model3 <- lm(mpg ~ poly(horsepower, 3), data = Auto)

# predict
hp_seq <- seq(min(Auto$horsepower), max(Auto$horsepower), length.out = 500)
pred1 <- predict(model1, newdata = data.frame(horsepower = hp_seq))
pred2 <- predict(model2, newdata = data.frame(horsepower = hp_seq))
pred3 <- predict(model3, newdata = data.frame(horsepower = hp_seq))
model_preds <- data.frame(
  horsepower = hp_seq,
  model1 = pred1,
```

```

model2 = pred2,
model3 = pred3
)

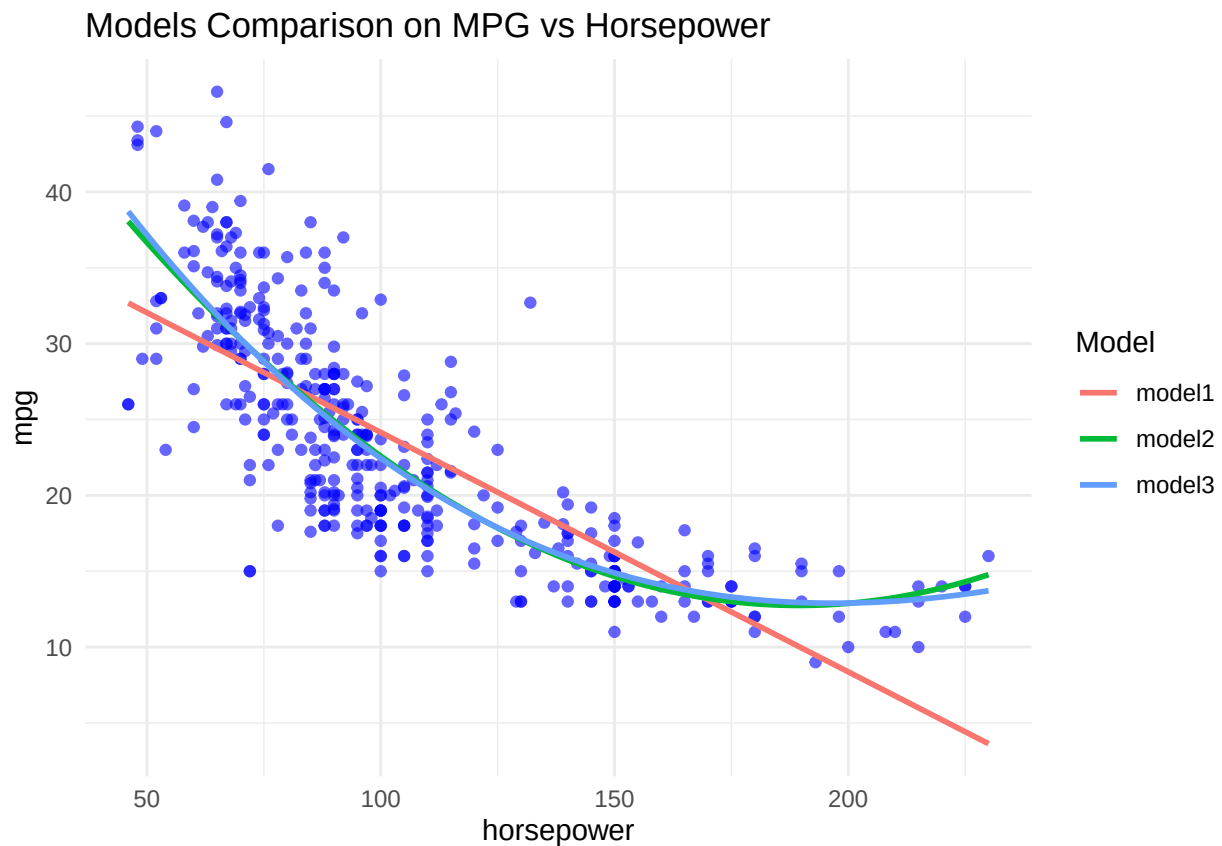
library(ggplot2)

## Warning: package 'ggplot2' was built under R version 4.4.2
library(tidyr)

## Warning: package 'tidyr' was built under R version 4.4.2
model_preds_long <- pivot_longer(model_preds, cols = -horsepower, names_to = "Model", values_to = "mpg")

# plot
ggplot(Auto, aes(x = horsepower, y = mpg)) +
  geom_point(color = "blue", alpha = 0.6) + # scatterplot
  geom_line(data = model_preds_long, aes(x = horsepower, y = mpg, color = Model), linewidth = 1) + # model lines
  labs(
    title = "Models Comparison on MPG vs Horsepower",
    x = "horsepower",
    y = "mpg",
    color = "Model"
  ) +
  theme_minimal()

```



2. Use the validation set approach to compare the models for 50/50 split and 70/30 split.

```
compute_metrics <- function(actual, predicted) {
  mse <- mean((actual - predicted)^2)
  rmse <- sqrt(mse)
  mad <- median(abs(actual - predicted))
  list(MSE = mse, RMSE = rmse, MAD = mad)
}
```

Split the data using a 50/50 split

```
# split data
n <- nrow(Auto)
split_ratio <- 0.5
train_indices <- sample(1:n, size = floor(split_ratio * n))
train_data <- Auto[train_indices, ]
test_data <- Auto[-train_indices, ]

# train
model1 <- lm(mpg ~ horsepower, data = train_data)
model2 <- lm(mpg ~ poly(horsepower, 2), data = train_data)
model3 <- lm(mpg ~ poly(horsepower, 3), data = train_data)
pred1 <- predict(model1, newdata = test_data)
pred2 <- predict(model2, newdata = test_data)
pred3 <- predict(model3, newdata = test_data)

# compute metrics
metrics1 <- compute_metrics(test_data$mpg, pred1)
metrics2 <- compute_metrics(test_data$mpg, pred2)
metrics3 <- compute_metrics(test_data$mpg, pred3)

metrics_df_50 <- data.frame(
  Model = c("Linear", "Quadratic", "Cubic"),
  MSE = c(metrics1$MSE, metrics2$MSE, metrics3$MSE),
  RMSE = c(metrics1$RMSE, metrics2$RMSE, metrics3$RMSE),
  MAD = c(metrics1$MAD, metrics2$MAD, metrics3$MAD)
)
kable(metrics_df_50, caption = "Model metrics comparison 50/50 split")
```

Table 2: Model metrics comparison 50/50 split

Model	MSE	RMSE	MAD
Linear	25.91434	5.090613	3.202879
Quadratic	20.60044	4.538771	2.675185
Cubic	20.55525	4.533790	2.669017

For the 50/50 split we can see based on the mse, rmse and mad that the best model is the cubic model because it has the lowest metrics

Split the data using a 70/30 split

```
# split data
n <- nrow(Auto)
split_ratio <- 0.7
train_indices <- sample(1:n, size = floor(split_ratio * n))
train_data <- Auto[train_indices, ]
test_data <- Auto[-train_indices, ]

# train
model1 <- lm(mpg ~ horsepower, data = train_data)
model2 <- lm(mpg ~ poly(horsepower, 2), data = train_data)
model3 <- lm(mpg ~ poly(horsepower, 3), data = train_data)
pred1 <- predict(model1, newdata = test_data)
pred2 <- predict(model2, newdata = test_data)
pred3 <- predict(model3, newdata = test_data)

# compute metrics
metrics1 <- compute_metrics(test_data$mpg, pred1)
metrics2 <- compute_metrics(test_data$mpg, pred2)
metrics3 <- compute_metrics(test_data$mpg, pred3)

metrics_df_70 <- data.frame(
  Model = c("Linear", "Quadratic", "Cubic"),
  MSE = c(metrics1$MSE, metrics2$MSE, metrics3$MSE),
  RMSE = c(metrics1$RMSE, metrics2$RMSE, metrics3$RMSE),
  MAD = c(metrics1$MAD, metrics2$MAD, metrics3$MAD)
)
kable(metrics_df_70, caption = "Model metrics comparison 70/30 split")
```

Table 3: Model metrics comparison 70/30 split

Model	MSE	RMSE	MAD
Linear	23.74293	4.872672	2.786518
Quadratic	20.47384	4.524803	2.709862
Cubic	20.49932	4.527618	2.673390

For the 70/30 split the best model is the Quadratic model because it has the lowest score on two out of the three metrics, and is followed closely in performance by the cubic model.

3. Use the `cv.glm` function in the `boot` package for leave one out cross validation and k fold cross validation

```
library(boot)

## Warning: package 'boot' was built under R version 4.4.2
model1 <- glm(mpg ~ horsepower, data = Auto)
model2 <- glm(mpg ~ poly(horsepower, 2), data = Auto)
model3 <- glm(mpg ~ poly(horsepower, 3), data = Auto)
```

Leave one out cross Validation

```

loocv_model1 <- cv.glm(Auto, model1)
loocv_model2 <- cv.glm(Auto, model2)
loocv_model3 <- cv.glm(Auto, model3)

loocv_results <- data.frame(
  Model = c("Linear", "Quadratic", "Cubic"),
  Error = c(loocv_model1$delta[1], loocv_model2$delta[1], loocv_model3$delta[1])
)
kable(loocv_results, caption = "Leave one out cross validation metrics comparison")

```

Table 4: Leave one out cross validation metrics comparison

Model	Error
Linear	24.23151
Quadratic	19.24821
Cubic	19.33498

In this scenario we can conclude that the best model would be the quadratic model

5-fold and 10-fold cross validation

```

kfold5_model1 <- cv.glm(Auto, model1, K = 5)
kfold5_model2 <- cv.glm(Auto, model2, K = 5)
kfold5_model3 <- cv.glm(Auto, model3, K = 5)

kfold10_model1 <- cv.glm(Auto, model1, K = 10)
kfold10_model2 <- cv.glm(Auto, model2, K = 10)
kfold10_model3 <- cv.glm(Auto, model3, K = 10)

kfold_results <- data.frame(
  Model = c("Linear", "Quadratic", "Cubic"),
  `5-Fold CV Error` = c(kfold5_model1$delta[1], kfold5_model2$delta[1], kfold5_model3$delta[1]),
  `10-Fold CV Error` = c(kfold10_model1$delta[1], kfold10_model2$delta[1], kfold10_model3$delta[1])
)
kable(kfold_results, caption = "K fold cross validation metrics comparison")

```

Table 5: K fold cross validation metrics comparison

Model	X5.Fold.CV.Error	X10.Fold.CV.Error
Linear	24.30410	24.10205
Quadratic	19.15354	19.41823
Cubic	19.22691	19.35220

In the case of 5-fold CV we can say that the best model is the quadratic model, but for 10-fold validation the best model would be the cubic model

4. Compare all results from 2 and 3. in a table and draw your conclusions.

```

kable(metrics_df_50, caption = "Model metrics comparison 50/50 split")

```

Table 6: Model metrics comparison 50/50 split

Model	MSE	RMSE	MAD
Linear	25.91434	5.090613	3.202879
Quadratic	20.60044	4.538771	2.675185
Cubic	20.55525	4.533790	2.669017

```
kable(metrics_df_70, caption = "Model metrics comparison 70/30 split")
```

Table 7: Model metrics comparison 70/30 split

Model	MSE	RMSE	MAD
Linear	23.74293	4.872672	2.786518
Quadratic	20.47384	4.524803	2.709862
Cubic	20.49932	4.527618	2.673390

```
kable(loocv_results, caption = "Leave one out cross validation metrics comparison")
```

Table 8: Leave one out cross validation metrics comparison

Model	Error
Linear	24.23151
Quadratic	19.24821
Cubic	19.33498

```
kable(kfold_results, caption = "K fold cross validation metrics comparison")
```

Table 9: K fold cross validation metrics comparison

Model	X5.Fold.CV.Error	X10.Fold.CV.Error
Linear	24.30410	24.10205
Quadratic	19.15354	19.41823
Cubic	19.22691	19.35220

Overall in most cases the best model is the Quadratic model, followed very closely in performance by the cubic model. The worst performing model was the linear model

Task 2

```
data(economics)
kable(head(economics), caption = "economics dataset (head)")
```

Table 10: economics dataset (head)

date	pce	pop	psavert	uempmed	unemploy
1967-07-01	506.7	198712	12.6	4.5	2944

date	pce	pop	psavert	uempmed	unemploy
1967-08-01	509.8	198911	12.6	4.7	2945
1967-09-01	515.6	199113	11.9	4.6	2958
1967-10-01	512.2	199311	12.9	4.9	3143
1967-11-01	517.4	199498	12.8	4.7	3066
1967-12-01	525.1	199657	11.8	4.8	3018

1. Fit multiple models

Linear models

```
lm1 <- lm(unemploy ~ uempmed, data = economics)
summary(lm1)
```

```
##
## Call:
## lm(formula = unemploy ~ uempmed, data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3005.2  -792.9  -109.8   931.1  3600.7
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2956.8      126.8    23.32  <2e-16 ***
## uempmed        559.3       13.3    42.06  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1307 on 572 degrees of freedom
## Multiple R-squared:  0.7557, Adjusted R-squared:  0.7553
## F-statistic: 1769 on 1 and 572 DF, p-value: < 2.2e-16
```

```
lm2 <- lm(uempmed ~ unemploy, data = economics)
summary(lm2)
```

```
##
## Call:
## lm(formula = uempmed ~ unemploy, data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -4.2674 -1.5802  0.0181  1.0254  7.5343
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -1.892e+00  2.637e-01  -7.177  2.22e-12 ***
## unemploy     1.351e-03  3.212e-05  42.064  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.032 on 572 degrees of freedom
## Multiple R-squared:  0.7557, Adjusted R-squared:  0.7553
## F-statistic: 1769 on 1 and 572 DF, p-value: < 2.2e-16
```

Exponential and logarithmic models

```
log_lm1 <- lm(log(unemploy) ~ uempmed, data = economics)
summary(log_lm1)

##
## Call:
## lm(formula = log(unemploy) ~ uempmed, data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.75125 -0.08753  0.02885  0.15297  0.40115
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  8.32073     0.02231   372.90  <2e-16 ***
## uempmed       0.06705     0.00234    28.66  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.23 on 572 degrees of freedom
## Multiple R-squared:  0.5894, Adjusted R-squared:  0.5887
## F-statistic: 821.2 on 1 and 572 DF, p-value: < 2.2e-16

log_lm2 <- lm(log(uempmed) ~ unemploy, data = economics)
summary(log_lm2)

##
## Call:
## lm(formula = log(uempmed) ~ unemploy, data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.37118 -0.13790  0.01958  0.12951  0.43541
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 1.044e+00  2.191e-02  47.63  <2e-16 ***
## unemploy     1.318e-04  2.670e-06  49.37  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1689 on 572 degrees of freedom
## Multiple R-squared:  0.8099, Adjusted R-squared:  0.8096
## F-statistic: 2438 on 1 and 572 DF, p-value: < 2.2e-16

exp_lm1 <- lm(uempmed ~ log(unemploy), data = economics)
summary(exp_lm1)

##
## Call:
## lm(formula = uempmed ~ log(unemploy), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.7856 -1.9608 -0.4871  0.7934 10.5946
```

```
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -69.6121      2.7317  -25.48  <2e-16 ***
## log(unemploy)   8.7909      0.3068   28.66  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.634 on 572 degrees of freedom
## Multiple R-squared:  0.5894, Adjusted R-squared:  0.5887
## F-statistic: 821.2 on 1 and 572 DF,  p-value: < 2.2e-16

exp_lm2 <- lm(unemploy ~ log(uempmed), data = economics)
summary(exp_lm2)

##
## Call:
## lm(formula = unemploy ~ log(uempmed), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2280.5  -891.0  -252.0   878.5  3133.5
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -4935.7      261.8  -18.85  <2e-16 ***
## log(uempmed)   6143.9      124.4   49.37  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1153 on 572 degrees of freedom
## Multiple R-squared:  0.8099, Adjusted R-squared:  0.8096
## F-statistic: 2438 on 1 and 572 DF,  p-value: < 2.2e-16
```

Polynomial models

```
# models for unemploy ~ uempmed
poly2_lm1 <- lm(unemploy ~ poly(uempmed, 2), data = economics)
summary(poly2_lm1)

##
## Call:
## lm(formula = unemploy ~ poly(uempmed, 2), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2606.1  -915.0  -181.6   953.3  2842.0
##
## Coefficients:
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7771.31      49.81  156.00  <2e-16 ***
## poly(uempmed, 2)1  54976.61    1193.47   46.06  <2e-16 ***
## poly(uempmed, 2)2 -12796.93    1193.47  -10.72  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 1193 on 571 degrees of freedom
## Multiple R-squared:  0.7966, Adjusted R-squared:  0.7959
## F-statistic: 1118 on 2 and 571 DF,  p-value: < 2.2e-16

poly3_lm1 <- lm(unemploy ~ poly(uempmed, 3), data = economics)
summary(poly3_lm1)

##
## Call:
## lm(formula = unemploy ~ poly(uempmed, 3), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2711.2   -904.5   -265.9    887.7   3396.6
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      7771.31      48.33 160.792 < 2e-16 ***
## poly(uempmed, 3)1  54976.61     1157.94  47.478 < 2e-16 ***
## poly(uempmed, 3)2 -12796.93     1157.94 -11.051 < 2e-16 ***
## poly(uempmed, 3)3   7003.62     1157.94   6.048 2.65e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1158 on 570 degrees of freedom
## Multiple R-squared:  0.8089, Adjusted R-squared:  0.8079
## F-statistic: 804.3 on 3 and 570 DF,  p-value: < 2.2e-16

poly10_lm1 <- lm(unemploy ~ poly(uempmed, 10), data = economics)
summary(poly10_lm1)

##
## Call:
## lm(formula = unemploy ~ poly(uempmed, 10), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2152.6   -820.4   -262.4    798.1   3897.7
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      7771.31      46.36 167.627 < 2e-16 ***
## poly(uempmed, 10)1  54976.61     1110.73  49.496 < 2e-16 ***
## poly(uempmed, 10)2 -12796.93     1110.73 -11.521 < 2e-16 ***
## poly(uempmed, 10)3   7003.62     1110.73   6.305 5.81e-10 ***
## poly(uempmed, 10)4  -6532.03     1110.73  -5.881 7.00e-09 ***
## poly(uempmed, 10)5   2910.36     1110.73   2.620 0.00902 **
## poly(uempmed, 10)6  -2355.61     1110.73  -2.121 0.03438 *
## poly(uempmed, 10)7   2425.47     1110.73   2.184 0.02940 *
## poly(uempmed, 10)8    661.13     1110.73   0.595 0.55193
## poly(uempmed, 10)9  -2363.17     1110.73  -2.128 0.03381 *
## poly(uempmed, 10)10  1047.93     1110.73   0.943 0.34585
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 1111 on 563 degrees of freedom
## Multiple R-squared:  0.8263, Adjusted R-squared:  0.8232
## F-statistic: 267.9 on 10 and 563 DF,  p-value: < 2.2e-16

# models for uempmed ~ unemploy
poly2_lm2 <- lm(uempmed ~ poly(unemploy, 2), data = economics)
summary(poly2_lm2)

##
## Call:
## lm(formula = uempmed ~ poly(unemploy, 2), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -6.1198 -1.0305 -0.0088  0.9332  5.6293
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      8.60871    0.07198  119.59  <2e-16 ***
## poly(unemploy, 2)1 85.45530    1.72461   49.55  <2e-16 ***
## poly(unemploy, 2)2 25.73821    1.72461   14.92  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.725 on 571 degrees of freedom
## Multiple R-squared:  0.8243, Adjusted R-squared:  0.8236
## F-statistic: 1339 on 2 and 571 DF,  p-value: < 2.2e-16

poly3_lm2 <- lm(uempmed ~ poly(unemploy, 3), data = economics)
summary(poly3_lm2)

##
## Call:
## lm(formula = uempmed ~ poly(unemploy, 3), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -5.9754 -1.0193 -0.0223  0.9950  5.4680
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      8.60871    0.07192  119.705  <2e-16 ***
## poly(unemploy, 3)1 85.45530    1.72298   49.597  <2e-16 ***
## poly(unemploy, 3)2 25.73821    1.72298   14.938  <2e-16 ***
## poly(unemploy, 3)3 -2.48410    1.72298   -1.442    0.15
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.723 on 570 degrees of freedom
## Multiple R-squared:  0.8249, Adjusted R-squared:  0.824
## F-statistic: 895 on 3 and 570 DF,  p-value: < 2.2e-16

poly10_lm2 <- lm(uempmed ~ poly(unemploy, 10), data = economics)
summary(poly10_lm2)
```

```
##
## Call:
## lm(formula = uempmed ~ poly(unemploy, 10), data = economics)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -7.7220 -0.9584  0.0484  0.9590  6.2248
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      8.60871    0.06881 125.114 < 2e-16 ***
## poly(unemploy, 10)1  85.45530    1.64850  51.838 < 2e-16 ***
## poly(unemploy, 10)2  25.73821    1.64850  15.613 < 2e-16 ***
## poly(unemploy, 10)3  -2.48410    1.64850  -1.507  0.1324
## poly(unemploy, 10)4  -8.81973    1.64850  -5.350 1.28e-07 ***
## poly(unemploy, 10)5  -7.05050    1.64850  -4.277 2.23e-05 ***
## poly(unemploy, 10)6  -2.00523    1.64850  -1.216  0.2243
## poly(unemploy, 10)7   3.77674    1.64850   2.291  0.0223 *
## poly(unemploy, 10)8   3.75639    1.64850   2.279  0.0231 *
## poly(unemploy, 10)9   0.28411    1.64850   0.172  0.8632
## poly(unemploy, 10)10  1.48176    1.64850   0.899  0.3691
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.648 on 563 degrees of freedom
## Multiple R-squared:  0.8417, Adjusted R-squared:  0.8389
## F-statistic: 299.3 on 10 and 563 DF, p-value: < 2.2e-16
```

2. Plot the models

Plot for unemploy ~ uempmed models

```
library(tidyr)

uempmed_seq <- seq(min(economics$uempmed), max(economics$uempmed), length.out = 500)
df <- data.frame(uempmed = uempmed_seq)

model_preds <- data.frame(
  uempmed = uempmed_seq,
  lm1 = predict(lm1, newdata = df),
  log_lm1 = predict(log_lm1, newdata = df),
  exp_lm2 = predict(exp_lm2, newdata = df),
  poly2_lm1 = predict(poly2_lm1, newdata = df),
  poly3_lm1 = predict(poly3_lm1, newdata = df),
  poly10_lm1 = predict(poly10_lm1, newdata = df)
)

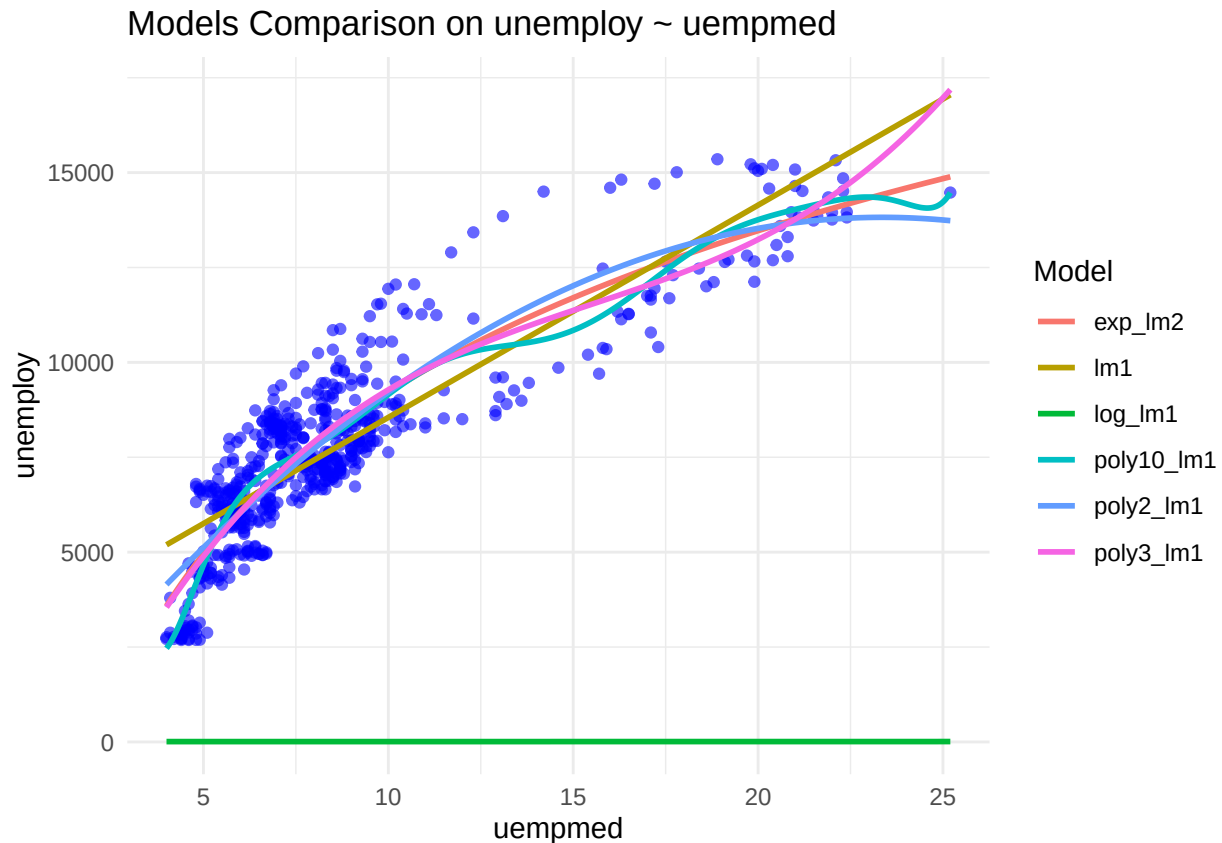
model_preds_long <- pivot_longer(model_preds, cols = -uempmed, names_to = "Model", values_to = "unemploy")

# Plot
ggplot(economics, aes(x = uempmed, y = unemploy)) +
  geom_point(color = "blue", alpha = 0.6) + # Scatter plot
  geom_line(data = model_preds_long, aes(x = uempmed, y = unemploy, color = Model), linewidth = 1) + #
  labs(
```

```

title = "Models Comparison on unemploy ~ uempmed",
x = "uempmed",
y = "unemploy",
color = "Model"
) +
theme_minimal()

```



Based on the graph i think the best model could be the exponential model

Plot for uempmed ~ unemploy models

```

library(tidyr)

unemploy_seq <- seq(min(economics$unemploy), max(economics$unemploy), length.out = 500)
df <- data.frame(unemploy = unemploy_seq)

model_preds <- data.frame(
  unemploy = unemploy_seq,
  lm2 = predict(lm2, newdata = df),
  log_lm2 = predict(log_lm2, newdata = df),
  exp_lm1 = predict(exp_lm1, newdata = df),
  poly2_lm2 = predict(poly2_lm2, newdata = df),
  poly3_lm2 = predict(poly3_lm2, newdata = df),
  poly10_lm2 = predict(poly10_lm2, newdata = df)
)

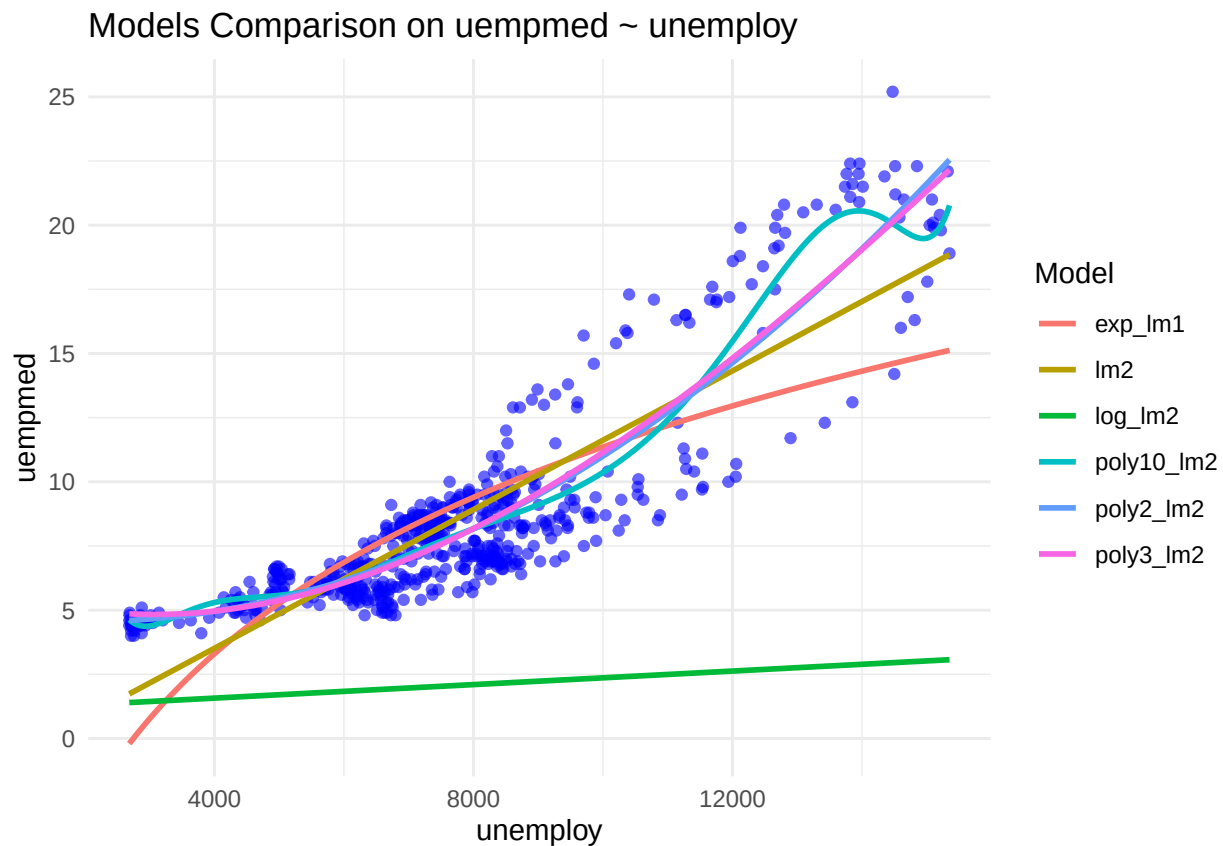
```

```

model_preds_long <- pivot_longer(model_preds, cols = -unemploy, names_to = "Model", values_to = "uempmed")

# Plot
ggplot(economics, aes(x = unemploy, y = uempmed)) +
  geom_point(color = "blue", alpha = 0.6) + # Scatter plot
  geom_line(data = model_preds_long, aes(x = unemploy, y = uempmed, color = Model), linewidth = 1) + #
  labs(
    title = "Models Comparison on uempmed ~ unemploy",
    x = "unemploy",
    y = "uempmed",
    color = "Model"
  ) +
  theme_minimal()

```



Based on the graph i think the best model could be the cubic polynomial model

3. Use the `cv.glm` function in the `boot` package

```

cv_rmse_mse <- function(model, data, K) {
  cv_result <- cv.glm(data, model, K = K)
  mse <- mean(cv_result$delta)
  rmse <- sqrt(mse)
  return(list(MSE = mse, RMSE = rmse))
}

models <- list(

```

```

lm1 = glm(unemploy ~ uempmed, data = economics),
lm2 = glm(uempmed ~ unemploy, data = economics),
log_lm1 = glm(log(unemploy) ~ uempmed, data = economics),
log_lm2 = glm(log(uempmed) ~ unemploy, data = economics),
exp_lm1 = glm(uempmed ~ log(unemploy), data = economics),
exp_lm2 = glm(unemploy ~ log(uempmed), data = economics),
poly2_lm1 = glm(unemploy ~ poly(uempmed, 2), data = economics),
poly3_lm1 = glm(unemploy ~ poly(uempmed, 3), data = economics),
poly10_lm1 = glm(unemploy ~ poly(uempmed, 10), data = economics),
poly2_lm2 = glm(uempmed ~ poly(unemploy, 2), data = economics),
poly3_lm2 = glm(uempmed ~ poly(unemploy, 3), data = economics),
poly10_lm2 = glm(uempmed ~ poly(unemploy, 10), data = economics)
)

```

LOOCV results

```

library(dplyr)

## Warning: package 'dplyr' was built under R version 4.4.2
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

cv_results_loo <- lapply(models, function(model) {
  cv_rmse_mse(model, economics, K = nrow(economics))
})

cv_results_df <- bind_rows(
  lapply(names(cv_results_loo), function(model_name) {
    result <- cv_results_loo[[model_name]]
    data.frame(
      Model = model_name,
      MSE = result$MSE,
      RMSE = result$RMSE
    )
  })
)
kable(cv_results_df, caption = "LOOCV results")

```

Table 11: LOOCV results

Model	MSE	RMSE
lm1	1.715205e+06	1309.6584020
lm2	4.159777e+00	2.0395530
log_lm1	5.311900e-02	0.2304756
log_lm2	2.861690e-02	0.1691654
exp_lm1	6.998820e+00	2.6455284

Model	MSE	RMSE
exp_lm2	1.333992e+06	1154.9858934
poly2_lm1	1.432524e+06	1196.8808579
poly3_lm1	1.366388e+06	1168.9259365
poly10_lm1	4.527867e+06	2127.8785961
poly2_lm2	3.005092e+00	1.7335201
poly3_lm2	3.009907e+00	1.7349084
poly10_lm2	2.832227e+00	1.6829222

We get the best performance with the logarithmic models

5 fold CV

```
cv_results_5fold <- lapply(models, function(model) {
  cv_result <- cv.glm(economics, model, K = 5) # 5-fold cross-validation
  mse <- mean(cv_result$delta)
  rmse <- sqrt(mse)
  return(list(MSE = mse, RMSE = rmse))
})

cv_results_df <- bind_rows(
  lapply(names(cv_results_5fold), function(model_name) {
    result <- cv_results_5fold[[model_name]]
    data.frame(
      Model = model_name,
      MSE = result$MSE,
      RMSE = result$RMSE
    )
  })
)
kable(cv_results_df, caption = "5 fold CV results")
```

Table 12: 5 fold CV results

Model	MSE	RMSE
lm1	1.713973e+06	1309.1878388
lm2	4.134215e+00	2.0332770
log_lm1	5.304910e-02	0.2303239
log_lm2	2.848100e-02	0.1687632
exp_lm1	7.042692e+00	2.6538071
exp_lm2	1.328317e+06	1152.5261500
poly2_lm1	1.433747e+06	1197.3916484
poly3_lm1	1.369634e+06	1170.3135825
poly10_lm1	1.191349e+07	3451.5922709
poly2_lm2	3.030607e+00	1.7408639
poly3_lm2	3.018901e+00	1.7374984
poly10_lm2	2.885017e+00	1.6985337

We get the best performance with the logarithmic models

10 fold CV

```
cv_results_10fold <- lapply(models, function(model) {  
  cv_result <- cv.glm(economics, model, K = 10) # 10-fold cross-validation  
  mse <- mean(cv_result$delta)  
  rmse <- sqrt(mse)  
  return(list(MSE = mse, RMSE = rmse))  
})  
  
cv_results_df <- bind_rows(  
  lapply(names(cv_results_10fold), function(model_name) {  
    result <- cv_results_10fold[[model_name]]  
    data.frame(  
      Model = model_name,  
      MSE = result$MSE,  
      RMSE = result$RMSE  
    )  
  })  
)  
kable(cv_results_df, caption = "5 fold CV results")
```

Table 13: 5 fold CV results

Model	MSE	RMSE
lm1	1.716989e+06	1310.3392749
lm2	4.145821e+00	2.0361288
log_lm1	5.318230e-02	0.2306129
log_lm2	2.853340e-02	0.1689184
exp_lm1	6.976323e+00	2.6412729
exp_lm2	1.331799e+06	1154.0360977
poly2_lm1	1.433283e+06	1197.1979985
poly3_lm1	1.371533e+06	1171.1247052
poly10_lm1	3.871447e+06	1967.5992498
poly2_lm2	3.027919e+00	1.7400917
poly3_lm2	2.995612e+00	1.7307835
poly10_lm2	2.809460e+00	1.6761444

We get the best performance with the logarithmic models and also some good performance with the polynomial models

4. Explain based on the CV and graphical model fits the concepts of Underfitting, Overfitting and how to apply cross-validation to determine the appropriate model fit.

Underfitting occurs when a model is too simple meaning it has not learned enough from the data or is too general, resulting in poor performance on both training and test data.

Overfitting occurs when a model is too complex meaning it has learned the training data too well and cannot generalize on unseen cases (test data), in other words performing with a very high accuracy on the training set but with a poor accuracy on the testing set.

Cross validation allows us to test the models on different subsets of the data and is a good way to prevent Underfitting and Overfitting. LOOCV leaves only one entry as testing data, while K fold CV leaves 1 fold out of the K folds as testing data and trains on the remaining K-1 folds. These methods were also very

usefull in figuring out way faster which model fits the data better by having the ability to compute the metrics we wanted very fast.